SEMESTER-VI

COURSE 14 A: NEURAL NETWORKS AND DEEP LEARNING

Theory **Credits: 3** **3 hrs/week**

Course Objectives:

1. Introduce the fundamental concepts of Artificial Neural Networks and Deep Learning, along with their historical and biological inspirations.
2. Provide an in-depth understanding of different neural network architectures including Perceptron, DNN, CNN, RNN, and advanced models.
3. Develop hands-on skills to design, train, and evaluate deep learning models using popular frameworks such as TensorFlow and Keras.
4. Expose students to applications of deep learning in computer vision, natural language processing, and generative modeling.
5. Enable students to critically analyze challenges in deep learning such as overfitting, bias, and ethical concerns.

Course Outcomes:

After successful completion of this course, students will be able to:

1. Explain the principles of neural networks, perceptrons, activation functions, and the evolution of deep learning.
2. Apply concepts of forward/backward propagation, weight initialization, and optimization techniques to train deep neural networks.
3. Design and implement convolutional neural networks (LeNet, AlexNet, VGG) for image classification tasks.
4. Build and analyze recurrent neural networks (RNN, LSTM, GRU) for sequential data and natural language processing applications.
5. Experiment with advanced deep learning concepts such as transfer learning, generative models, and transformers using pre-trained models.

Unit 1. Foundations of Deep Learning:

What is Artificial Intelligence, Machine Learning, and Deep Learning? History and applications of deep learning, Biological vs. Artificial Neurons

Introduction to Neural Networks, Perceptron and activation functions (Linear, ReLU, Sigmoid, Tanh, Softmax), Types of Neural Networks (shallow vs. deep, feedforward vs. recurrent), Gradient descent and backpropagation (conceptual only), Concept of loss functions (MSE, cross-entropy) at intuitive level

Unit 2. Deep Neural Networks:

Forward and backward propagation, Weight initialization, learning rate, and optimization algorithms (SGD, Adam, RMSProp), Overfitting & underfitting: Regularization, Dropout, Batch normalization, Activation functions in deep networks, Loss functions in detail (MSE, cross-entropy, hinge loss)
Introduction to Keras/TensorFlow framework

Unit 3. Convolutional Neural Networks (CNNs):

Introduction to images and pixels, Filters/kernels, padding, and pooling, CNN architecture and layers (Conv, Pooling, Fully Connected, Softmax), Classical CNN architectures: LeNet-5 (digit recognition - first CNN model), AlexNet (ImageNet breakthrough - deeper CNN), VGG (concept of depth, simplicity)
Applications in image classification, object detection, facial recognition

Unit 4. Recurrent Neural Networks (RNNs) and NLP

Sequences and time series data, Introduction to RNNs: vanishing/exploding gradient issue LSTM and GRU (intuitive and architectural view), Word embeddings: Word2Vec, GloVe, introduction to contextual embeddings (BERT at high level)
Applications: Sentiment analysis, text generation, simple time-series forecasting

Unit 5. Advanced & Emerging Topics:

Generative models: GANs (Generator & Discriminator intuition), VAEs (introduction only), Transformers: attention mechanism (intuitive), BERT, GPT family (overview), Transfer learning & fine-tuning pre-trained models (vision & NLP), AI ethics: Bias, fairness, privacy, safety, explainability

Textbooks:

1. Chollet, F. (2018). *Deep Learning with Python* (1st ed.). Manning Publications.
2. Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press. *(Available free online: http://neuralnetworksanddeeplearning.com)*

Reference Books:

1. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.

2. Howard, J., & Gugger, S. (2020). *Practical Deep Learning for Coders*. O'Reilly Media. *(Based on the fast.ai course)*
3. Shane, J. (2019). *You Look Like a Thing and I Love You: How AI Works and Why It's Making the World a Weirder Place*. Voracious / Hachette Book Group.

Activities:

Outcome: Explain the principles of neural networks, perceptrons, activation functions, and the evolution of deep learning.

Activity: *Concept Mapping Exercise* - Students work in small groups to draw a timeline-based concept map linking biological neurons → perceptron → multilayer perceptron → activation functions → deep learning.

Evaluation Method: Rubric-based evaluation of concept maps (clarity, correctness, and depth of connections).

Outcome: Apply concepts of forward/backward propagation, weight initialization, and optimization techniques to train deep neural networks.

Activity: *Hands-on Coding Task* - Students implement a simple 3-layer neural network from scratch in Python/NumPy (without Keras/TensorFlow) to observe propagation and optimization effects.

Evaluation Method: Code demonstration + oral viva where students explain how changing initialization/learning rate affects training.

Outcome: Design and implement convolutional neural networks (LeNet, AlexNet, VGG) for image classification tasks.

Activity: *Mini Project (Image Classification)* - Students form teams to train CNNs (LeNet, AlexNet, VGG) on a dataset like MNIST/CIFAR-10 and compare results.

Evaluation Method: Project report + presentation with accuracy comparison, confusion matrix, and discussion of design choices.

Outcome: Build and analyze recurrent neural networks (RNN, LSTM, GRU) for sequential data and natural language processing applications.

Activity: *Case Study on Sentiment Analysis* - Students use IMDB reviews dataset to build RNN/LSTM/GRU models for sentiment classification and analyze performance.

Evaluation Method: Submission of case study report with experimental setup, evaluation metrics (accuracy/F1-score), and reflection on differences among models.

Outcome: Experiment with advanced deep learning concepts such as transfer learning, generative models, and transformers using pre-trained models.

Activity: *Model Exploration & Demonstration* - Students choose one advanced technique (Transfer Learning on ResNet, GAN for image generation, or Transformer for text classification) and prepare a live demo in class.

Evaluation Method: Evaluation of demo + short reflective note (1–2 pages) on challenges, benefits, and application potential of the chosen technique.

SEMESTER-VI

COURSE 14 A: NEURAL NETWORKS AND DEEP LEARNING

Practical **Credits: 1** **2 hrs/week**

1. Build a perceptron from scratch in Python
2. Use Google Teachable Machine or Tensor Flow Playground
3. Visualize various Activation Functions and their Gradients
4. Build and train a deep neural network for classification (e.g., MNIST digits)
5. Experiment with dropout, batch normalization, and different activations
6. Train a CNN to classify fashion images (Fashion-MNIST)
7. Visualize filters and feature maps
8. Fine-tune a pre-trained CNN (Mobile Net, VGG) for a small dataset
9. Build an LSTM model for movie review sentiment analysis (IMDb dataset)
10. Generate text using a simple character-level RNN
11. Use a pre-trained model (like MobileNet or BERT) for a simple task
12. Use Huggingface to deploy a Sentiment Analysis App for Swiggy Reviews

SEMESTER-VI

COURSE 14 B: TIME SERIES ANALYSIS AND FORECASTING

Theory **Credits: 3** **3 hrs/week**

Course Objectives

The course aims to:

1. Provide fundamental understanding of time series data, components, and characteristics.
2. Train students in identifying, modeling, and forecasting using ARMA/ARIMA/SARIMA models.
3. Introduce state-space and multivariate approaches for complex data.
4. Familiarize students with modern forecasting methods, including spectral and evaluation techniques.
5. Enable hands-on practice with real-world datasets using R/Python statistical libraries.

Course Outcomes

By the end of the course, students will be able to:

1. Explain the concepts of time series, stationarity, and autocorrelation functions.
2. Apply ARMA/ARIMA/SARIMA models to real-world time series data.
3. Analyze multivariate and state-space time series using appropriate tools.
4. Implement forecasting workflows using R/Python for financial, business, and scientific datasets.
5. Evaluate forecast accuracy and select appropriate models using statistical criteria.

Unit 1. Fundamentals & Stationary Processes

Introduction to time series: types, components, forecasting process. Stationary processes: definitions, autocovariance, autocorrelation functions (ACF/PACF).

Model evaluation metrics. ACF/PACF example analyses.

Unit 2. ARMA & Forecasting with ARMA

ARMA(p,q) models: definition, estimation, forecasting approaches. Model identification: AIC, PACF/ACF, diagnostic checks. Practical examples of fitting ARMA and generating forecasts.

Unit 3. Non-Stationary & Seasonal Models

Non-stationary time series: differencing, unit roots. Seasonal models: SARIMA and multiplicative seasonal ARIMA. Identification, estimation, and diagnostic checks for seasonal models.

Unit 4. State-Space & Multivariate Time Series

Multivariate time series: Vector ARMA models (VARMA), estimation, forecasting. State-space representation: formulation, Kalman filter basics, forecasting in state-space models.

Unit 5. Advanced Topics & Forecast Evaluation

Spectral analysis: frequency-domain representation, spectral density. Forecast performance: measures, monitoring, choosing models.

Textbook:

1. Introduction to Time Series and Forecasting, Peter J. Brockwell & Richard A. Davis, 2nd Edition, Springer

Reference Books

1. Time Series Analysis: Forecasting and Control, George E. P. Box, Gwilym M. Jenkins & Gregory C. Reinsel
2. Introduction to Time Series Analysis and Forecasting, Douglas C. Montgomery, Cheryl L. Jennings, Murat Kulahci , (Wiley)
3. Time Series Analysis and Its Applications: With R Examples, R. H. Shumway & D. S. Stoffer

Activities:

Outcome: Explain the concepts of time series, stationarity, and autocorrelation functions

Activity: Students will prepare a seminar or short presentation explaining stationarity, ACF, PACF with a simple dataset example (like sales data).

Evaluation Method: Evaluated through presentation quality, understanding during viva, and a short concept quiz.

Outcome: Apply ARMA/ARIMA/SARIMA models to real-world time series data

Activity: Students will conduct a hands-on lab task to fit ARIMA and SARIMA models on stock price or rainfall data using Python/R.

Evaluation Method: Lab record submission, correctness of implementation, and a practical exam.

Outcome: Analyze multivariate and state-space time series using appropriate tools
Activity: Students will carry out a case study on macroeconomic datasets (like GDP, inflation, unemployment) using VAR or state-space modeling.
Evaluation Method: Case study report, results interpretation, and oral viva.

Outcome: Implement forecasting workflows using R/Python for financial, business, and scientific datasets
Activity: Students will design an end-to-end forecasting pipeline (data preprocessing → model building → forecasting → visualization). Example: forecasting COVID-19 daily cases or retail sales.
Evaluation Method: Project demo, code submission, and project report.

Outcome: Evaluate forecast accuracy and select appropriate models using statistical criteria
Activity: Students will compare multiple forecasting methods (e.g., ARIMA vs. Exponential Smoothing) on the same dataset and analyze performance using RMSE, MAE, and MAPE.
Evaluation Method: Written assignment, interpretation of metrics, and justification of chosen model.

SEMESTER-VI

COURSE 14 B: TIME SERIES ANALYSIS AND FORECASTING

Practical **Credits: 1** **2 hrs/week**

(Using R/Python statsmodels, pandas, forecast, or equivalent)

1. Import and visualize time series datasets (stock, weather, sales).
2. Perform decomposition of time series into trend, seasonal, residual components.
3. Compute and plot Autocorrelation Function (ACF) & Partial ACF (PACF).
4. Test stationarity using Augmented Dickey-Fuller (ADF) test.
5. Fit ARMA models and validate residuals.
6. Implement ARIMA and SARIMA models for seasonal data.
7. Perform model selection using AIC/BIC and cross-validation.
8. Forecast with ARIMA/SARIMA and plot prediction intervals.
9. Apply multivariate time series (VAR) to macroeconomic datasets.
10. Explore state-space models using Kalman filtering.
11. Conduct spectral analysis of a time series.
12. Compare forecasting methods: ARIMA vs. Exponential Smoothing vs. ML models.
13. Evaluate forecast performance with RMSE, MAPE, etc.

SEMESTER-VI

COURSE 15 A: NATURAL LANGUAGE PROCESSING

Theory **Credits: 3** **3 hrs/week**

Course Objectives:

1. Introduce the foundations of Natural Language Processing and its applications in real-world tasks.
2. Familiarize students with text preprocessing, linguistic analysis, and parsing techniques.
3. Equip learners with methods for information extraction, word representations, and sentiment classification.
4. Explore deep learning techniques for NLP, including RNNs, LSTMs, GRUs, and Transformers.
5. Provide hands-on experience with modern NLP tools (NLTK, spaCy, Hugging Face) for implementing applications such as chatbots, summarization, and document classification.

Course Outcomes:

At the end of this course, students will be able to:

1. Explain the principles, challenges, and applications of NLP and use basic text processing tools.
2. Apply preprocessing techniques (tokenization, stemming, lemmatization) and parsing methods to analyze language structures.
3. Implement information extraction and text representation methods (NER, embeddings, classification pipelines).
4. Build and evaluate deep learning models (RNN, LSTM, GRU, Transformer) for NLP tasks.
5. Utilize pre-trained transformer models (BERT, GPT) with Hugging Face for advanced NLP applications such as summarization, chatbots, and document classification.

Unit 1. Introduction to NLP and Language Fundamentals:

Definition, Goals, and Scope of NLP, Real-world Applications (Assistants, Chatbots, Translation, Summarization, QA, Spam Detection), Fundamentals of Language Processing, Ambiguities in NLP (Lexical, Structural, Contextual)

Installations: Python setup, NLTK, spaCy basics

Regular Expressions (Essential patterns, findall, split, sub, matching tokens)

Unit 2. Text Preprocessing and Linguistic Analysis:

Key NLP Terminologies: Morphology, Lexicon, Orthographic Rules

Finite State Transducers

Text Preprocessing Techniques: Tokenization, Stopword Removal, Stemming, Lemmatization

Grammar and Context-Free Grammar

Parsing Techniques: Top-down, Bottom-up, CYK Algorithm

Semantic Analysis: Elements, Meaning Representation

Unit 3. Information Extraction and Representation:

Named Entity Recognition (NER): Concepts, Examples, Using spaCy & NLTK

Word Embeddings: Word2Vec (Skip-Gram, CBOW), Comparison, Implementations, Bag of Words and N-grams, Text Classification Pipeline, Sentiment Analysis Applications

Ethical considerations in preprocessing & classification

Unit 4. Deep Learning for NLP:

Recurrent Neural Networks (RNN): Basics, RNN vs CNN/Feedforward NN, LSTM and GRU for Sequence Modeling, Transformer Models: Introduction, Pretrained Models (BERT, GPT), Hugging Face Ecosystem

Unit 5. Transformers and Modern NLP:

Transformer architecture basics (self-attention, encoder-decoder), BERT: Pretraining, Fine-tuning,

GPT and Generative NLP, Hugging Face Ecosystem (using pre-trained models)

Text Summarization: Extractive, Abstractive, Hybrid Approaches

Applications: Document Classification, Chatbots, Virtual Assistants

Textbook:

1. Natural Language Processing, Sini Raj Pulari, Umadevi Maramreddy, Shriram k. Vasudevan, Oxford University Press
2. Speech and Language Processing, An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, Daniel Jurafsky, James H. Martin, Pearson Education, 2023.

Reference Book:

1. Natural Language Processing and Information Retrieval, Tanveer Siddiqui, U.S. Tiwary , Oxford University Press.
2. 2. Natural Language Processing Recipes - Unlocking Text Data with Machine Learning and Deep Learning using Python, Akshay Kulkarni, Adarsha Shivananda, Apress, 2019.

Activities:

Outcome: Explain NLP fundamentals and basic text processing tools.

Activity: Quiz/ Assignment on Analyze ambiguities in Indian language sentences.

Evaluation Method: Quiz Score

Outcome: Apply preprocessing and parsing techniques to analyze language.

Activity: Case study: Building a simple grammar-based sentence parser.

Evaluation Method: Application of text preprocessing to raw text corpus, parsing

Outcome: Implement information extraction and text representation methods.

Activity: Hands-on NER using spaCy and NLTK.

Evaluation Method: Lab report submission on embeddings & NER.

Outcome: Build and evaluate deep learning models for NLP.

Activity: Group Discussion on Discussion: Compare RNN vs Transformers.

Evaluation Method: Depth of Understanding, Participation, Explanation

Outcome: Utilize pre-trained transformer models for advanced NLP applications.

Activity: Lab: Implement chatbot using GPT model.

Evaluation Method: Practical exam using Hugging Face models – Accuracy, Effectiveness

SEMESTER-VI

COURSE 15 A: NATURAL LANGUAGE PROCESSING

Practical **Credits: 1** **2 hrs/week**

1. Install Python, NLTK, and spaCy. Write a sample program to print available NLP corpora and models.
2. Write regex patterns for extracting emails, phone numbers, hashtags, and dates from a text file.
3. Demonstrate lexical and structural ambiguity with example sentences. Use NLTK parse trees to visualize.
4. Implement sentence and word tokenization using NLTK and spaCy. Compare outputs.
5. Write a program to remove stopwords and analyze text length reduction.
6. Apply Stemming and Lemmatization on a dataset and compare differences.
7. Use NLTK to demonstrate top-down and bottom-up parsing of a simple grammar.
8. Use spaCy to extract entities (e.g., names, locations, organizations) from news text.
9. Implement text representation and calculate similarity between documents.(Bag of Words and N-grams)
10. Build a sentiment classifier using Scikit-learn (Naive Bayes / Logistic Regression).
11. Implement a simple RNN to generate sentences character by character.
12. Hugging Face to load a pretrained BERT model and perform masked word prediction.
13. Implement extractive and abstractive summarization using Hugging Face pipelines.
14. Build a simple FAQ-based chatbot using Transformer-based embeddings.

SEMESTER-VI

COURSE 15 B: DATA ENGINEERING & MLOPS

Theory **Credits: 3** **3 hrs/week**

Course Objectives

1. To introduce the lifecycle and roles in Data Engineering.
2. To explore data architecture principles, distributed systems, and technology choices.
3. To analyze MLOps features, risks, and challenges in developing ML systems.
4. To design CI/CD pipelines and deployment strategies for ML models.
5. To understand monitoring, governance, and Responsible AI compliance in production ML.

Course Outcomes

At the end of the course, students will be able to:

1. Explain Data Engineering and its organizational roles.
2. Analyze major concepts in data architecture and distributed systems.
3. Apply MLOps features and evaluate challenges in ML model development.
4. Design and implement CI/CD pipelines for ML deployment.
5. Evaluate governance and Responsible AI practices in MLOps.

Unit 1. Foundations of Data Engineering

Data Engineering: definition, lifecycle, skills, activities. Evolution and roles of Data Engineers: technical vs business responsibilities, internal vs external roles. Relationship between Data Engineering and Data Science. Data lifecycle vs Data Engineering lifecycle.

Unit 2. Data Architecture & Distributed Systems

Enterprise and Data Architecture definitions. Principles of good data architecture. Scalability, failure design, tiers, microservices, monolith vs modular. Event-driven architecture, hybrid cloud, multicloud, edge computing. Technology selection criteria: team size, interoperability, cost, TCO.

Unit 3. MLOps Fundamentals

MLOps challenges and risk mitigation. Responsible AI and scaling ML solutions. Key MLOps features: EDA, feature engineering, model training & evaluation, reproducibility. Deployment requirements, monitoring basics. Model versioning and experimentation tracking.

Unit 4. Model Deployment & CI/CD Pipelines

Preparing models for production. Runtime environments: dev to production adaptation.
CI/CD pipelines: building ML artifacts, testing pipelines. Deployment strategies: batch, online, A/B testing, canary releases. Containerization & scaling (Docker, Kubernetes).

Unit 5. Monitoring, Feedback Loops & Governance

Monitoring models in production: drift detection, ground truth evaluation.
Feedback loops: retraining workflows, online evaluation. Logging, monitoring frameworks.
Governance: regulations (GDPR, CCPA, GxP), Responsible AI principles.
Templates for governance, compliance, and model risk management.

Text/ Reference books

1. Fundamentals of Data Engineering, Joe Reis & Matt Housley, O'Reilly, 2022.
2. Introducing MLOps, Mark Treveil & Dataiku Team, O'Reilly, 2020

Web Resources:

https://www.ibm.com/think/topics/data-engineering
https://martinfowler.com/articles/microservices.html
https://towardsdatascience.com/a-gentle-introduction-to-mlops-7d64a3e890ff/

Activities

Outcome: Explain Data Engineering and roles
Activity: Prepare a concept map showing different Data Engineer roles in an organization.
Evaluation Method: Short presentation + written quiz.

Outcome: Analyze data architecture concepts
Activity: Case study on choosing between monolith, microservices, and event-driven architectures.
Evaluation Method: Case study report + viva.

Outcome: Apply MLOps features in ML development

Activity: Lab exercise on feature engineering & reproducibility using MLflow.

Evaluation Method: Lab record submission + demo.

Outcome: Design CI/CD pipelines for ML

Activity: Mini-project: build a simple ML CI/CD pipeline with GitHub Actions/Docker.

Evaluation Method: Project demo + evaluation rubric.

Outcome: Evaluate governance and Responsible AI practices

Activity: Group discussion & policy brief on GDPR/Responsible AI practices.

Evaluation Method: Written assignment + peer review.

SEMESTER-VI

COURSE 15 B: DATA ENGINEERING & MLOPS

Practical **Credits: 1** **2 hrs/week**

1. Install and configure a modern data engineering environment (Python, Jupyter, VSCode).
2. Explore and visualize the data lifecycle on a sample dataset (sales/weather).
3. ETL basics: extract data from CSV/JSON -> transform -> load into relational database.
4. Compare performance of batch vs event-driven ingestion using Apache Kafka or RabbitMQ.
5. Deploy a small dataset on Hadoop Distributed File System (HDFS) and perform simple operations.
6. Case study lab: design a microservices vs monolithic workflow for a mock business problem.
7. Perform Exploratory Data Analysis (EDA) and track experiments using **MLflow**.
8. Build a simple ML model (regression/classification) and enable **reproducibility** with version control.
9. Manage datasets and model versions using **DVC (Data Version Control)**.
10. Containerize an ML model with **Docker**.
11. Automate training and deployment with a **GitHub Actions CI/CD pipeline**.
12. Deploy an ML model as a REST API using **FastAPI / Flask**.
13. Implement model drift detection: monitor incoming data and compare with training data distribution.
14. Build a simple feedback loop: retrain a model automatically when drift exceeds a threshold.
15. Configure logging and monitoring using **Prometheus/Grafana** for a deployed ML model.
16. Case study: analyze GDPR/Responsible AI implications on a real dataset (e.g., facial recognition).